Grounded RAG Prompt: Actual Structure

Exact authoritative implementation: core/rag/prompt_builder.py (../core/rag/prompt_builder.py).

• Trusted system message contains AUTOSAR engineering rules and prohibits inventing software components, port relationships, signals, runnables and other facts.

• Untrusted PDF content is enclosed in <context> and per-evidence <chunk id="..." doc="..." page="..." section="..."> elements.

• User question is isolated in <user_question>.

• All factual claims must cite the exact retrieved [chunk_id].

• Missing evidence requires: Based on the provided AUTOSAR specification, there is insufficient evidence to answer this question.

• The builder escapes certain XML-like delimiter sequences, reducing simple document-context breakout attacks (not a complete injection defense).

• Context accumulation is capped at 12,000 characters by default.

Illustrative interaction:

Question: “What does DoorControlSWC do?”

Relevant source: “DoorControlSWC processes lock/unlock requests and publishes door status.”

Desired generated output: “DoorControlSWC processes lock and unlock requests and publishes door status [ACTUAL_RETRIEVED_CHUNK_ID].”

ACTUAL_RETRIEVED_CHUNK_ID is a placeholder, not a valid live citation. Never fabricate IDs in an evaluation.

Real limitation: MockLLM has previously generated generic citation-bearing answers that were not fully responsive and failed to refuse an unsupported MCU question. Semantic claim support must be checked manually; this behavior is not a passed hallucination test.

Token management: reuse local embedding model, default top-k 5 and MockLLM for deterministic tests; send small retrieved context to Gemini only when needed.